UNIVERSIDAD DE LA SALLE
ESCUELA DE CIENCIAS BÁSICAS
PROGRAMA DE CIENCIA DE DATOS

Manual de Programación y Preubas - Televentas

Presentado por:
Sebastián Gutiérrez Guayacundo

Docente:
RODRIGO ARANDA FERNANDEZ

Lenguaje de Programación II
Semestre III – 2026
Bogotá, Colombia

Contenido

1. Enunciado del Problema: 3
2. Problema Resumido para el Código 3
3. Mapa UML de Código 4
4. Creación de Ramas y Repositorio 5
5. Algoritmo en Java 6
6. Código en Visual Studio Code 11
7. Prueba de Escritorio 16
Resultado: 16

1. Enunciado del Problema:

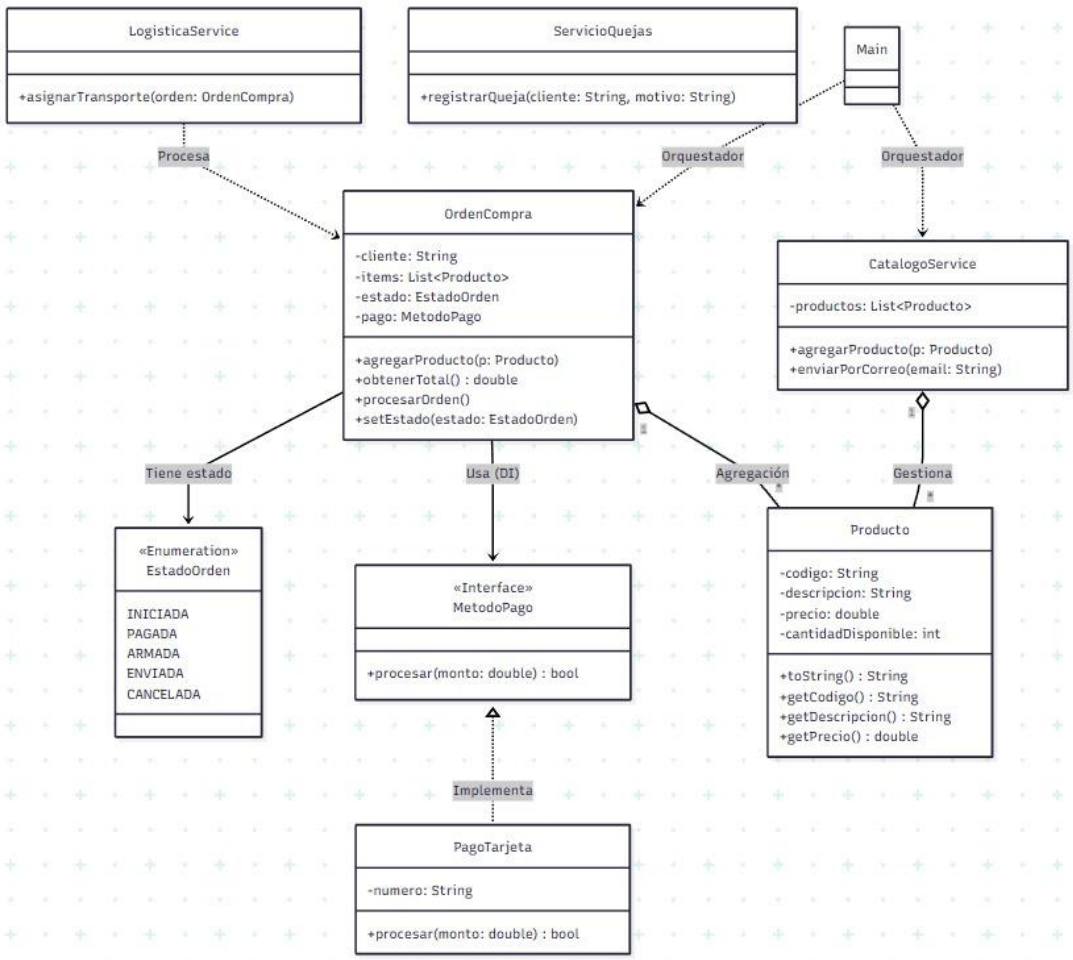
La empresa TeleVentas desea desarrollar un sistema web para el soporte de compras a distancia, de los productos que la empresa ofrece, por parte de sus clientes. Se prevé que, con este sistema, los clientes podrán, fundamentalmente: realizar consultas del catálogo de los productos que se ofrecen, para obtener información acerca de un producto particular (código, descripción, precio, cantidad disponible, etc.); solicitar le envíen periódicamente y por correo electrónico dicho catálogo de productos; ingresar una orden de compra, para que se le envíe un conjunto de productos, ajustándose a un tipo de pago para la misma (actualmente sólo tarjeta de crédito); presentar una queja (por demoras en la entrega de los productos, por ejemplo); y cancelar una orden.

Adicionalmente, se pretende que los agentes del depósito de la empresa utilicen el nuevo sistema, consultando las órdenes de compra confirmadas, para armar y empaquetar los productos pedidos en dichas órdenes de compra. Asimismo, se debe proveer soporte a estos agentes para determinar la logística para la entrega de cada pedido armado, seleccionando una empresa de transporte apropiada, y delegando a ésta la entrega del pedido correspondiente. Por otra parte, es destacable que la empresa ya posee un sistema de inventario para el control de stock de productos. El nuevo sistema deberá interactuar con éste para, por ejemplo: consultar descripción y precio de productos al tomar órdenes de compra, o bien para actualizar la disponibilidad de productos al armar pedidos. Por otra parte, las quejas recibidas por parte de los clientes son remitidas inmediatamente al gerente de relaciones de la empresa.

2. Problema Resumido para el Código

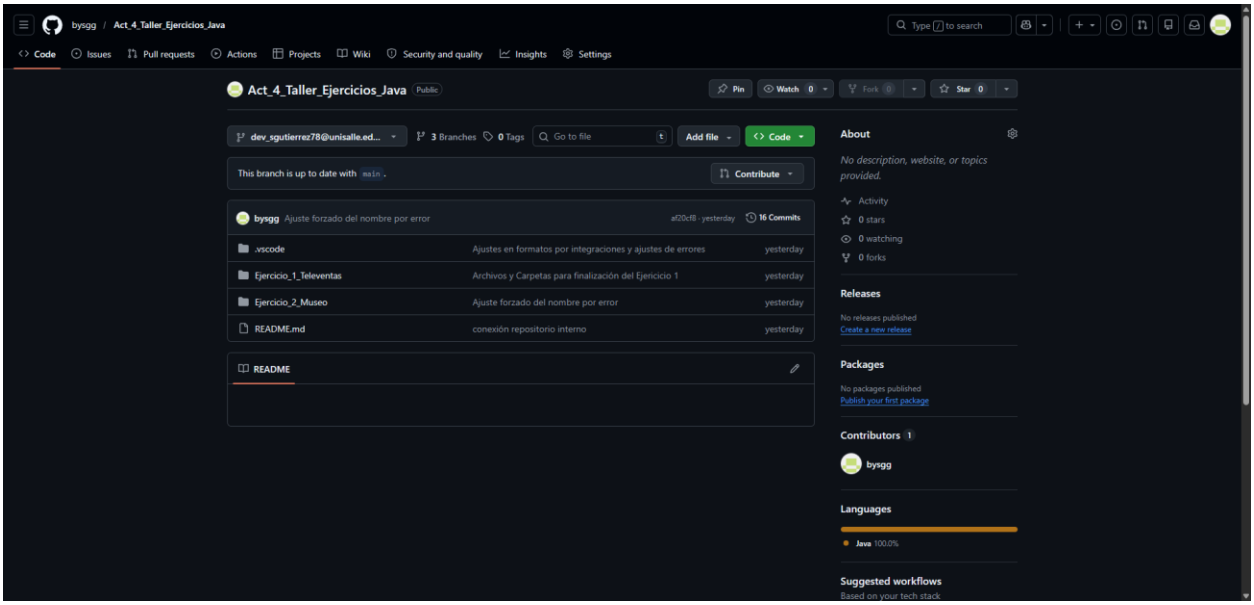
La empresa "TeleVentas" requiere un sistema robusto para gestionar ventas a distancia. El problema técnico principal radica en la sincronización de procesos distribuidos: un cliente inicia una orden, pero el stock debe apartarse inmediatamente para evitar ventas de productos inexistentes. Además, el flujo debe ser estrictamente secuencial; no se puede enviar un producto (Logística) si el depósito no lo ha empacado, y el depósito no puede empacar si la pasarela de pagos (Tarjeta de Crédito) no ha confirmado la transacción. Finalmente, el sistema debe ser capaz de desviar incidencias (Quejas) directamente a la alta gerencia sin interrumpir el flujo de ventas.

3. Mapa UML de Código



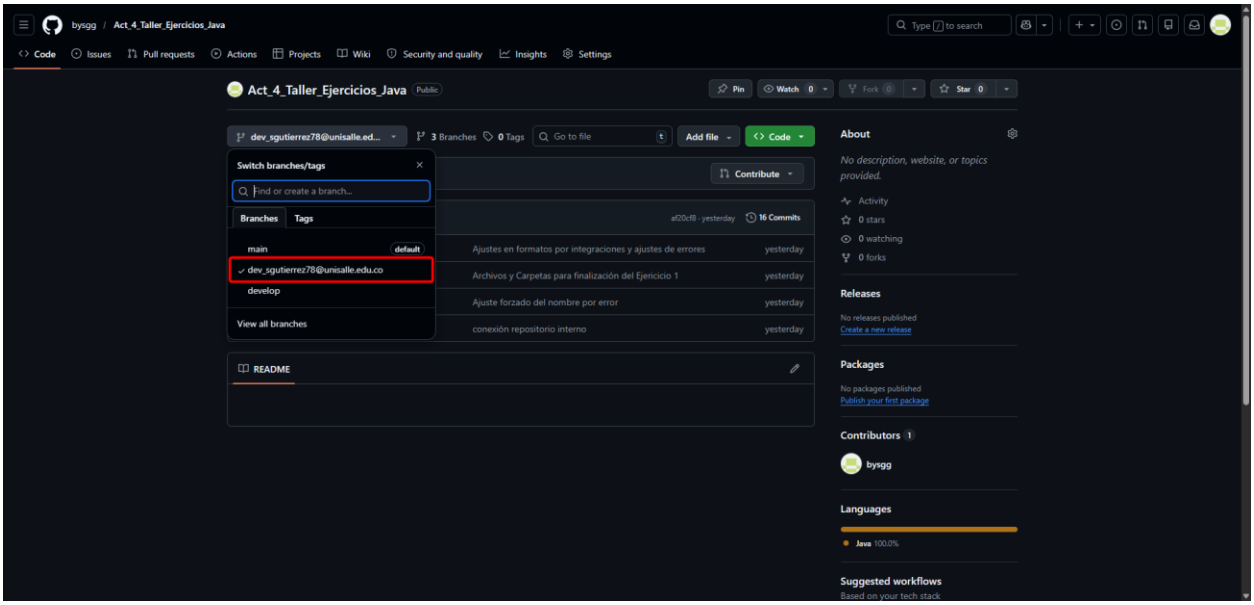
4. Creación de Ramas y Repositorio

https://github.com/bysgg/Act_4_Taller_Ejercicios_Java/tree/dev_sgutierrez78%40unisalle.edu.co



Se crean las ramas

- **Main, Develop y dev_sgutierrez78@unisalle.edu.co**



5. Algoritmo en Java

Main.java

```java
package Ejercicio_1_Televentas.com.televentas;

public class Main {
   public static void main(String[] args) {
      System.out.println("=== SISTEMA TELEVENTAS V1.0 ===");

      // 1. Setup del Catálogo
      CatalogoService catalogo = new CatalogoService();
      Producto p1 = new Producto("A01", "Laptop Gamer", 3500.0, 10);
      Producto p2 = new Producto("B02", "Mouse Optico", 50.0, 50);
      catalogo.agregarProducto(p1);
      catalogo.agregarProducto(p2);

      // 2. Cliente consulta y pide catálogo por correo
      System.out.println("\n[CLIENTE] Consultando catálogo...");
      catalogo.enviarPorCorreo("sebastian@lasalle.edu.co");

      // 3. Crear Orden de Compra
      MetodoPago tarjeta = new PagoTarjeta("1234-5678-9012-3456");
      OrdenCompra miOrden = new OrdenCompra("Sebastian", tarjeta);
      miOrden.agregarProducto(p1);
      miOrden.agregarProducto(p2);

      System.out.println("Total de la orden: $" + miOrden.obtenerTotal());

      // NUEVO: Procesar el pago antes de armar
      miOrden.procesarOrden();

      // 4. Agente de Depósito (Armar)
      System.out.println("\n[DEPOSITO] Armando pedido...");
      if (miOrden.getEstado() == EstadoOrden.PAGADA) {
         miOrden.setEstado(EstadoOrden.ARMADA);
      }

      // 5. Simular una Queja
      ServicioQuejas quejas = new ServicioQuejas();
      quejas.registrarQueja("Sebastian", "El paquete llegó con la caja abollada.");

      System.out.println("\n=== PROCESO FINALIZADO CON ÉXITO ===");
   }
}
```

CatalogoService.java

```java
package Ejercicio_1_Televentas.com.televentas;
```

```java
import java.util.ArrayList;
import java.util.List;

public class CatalogoService {
   private List<Producto> productos = new ArrayList<>();

   public void agregarProducto(Producto p) {
      productos.add(p);
   }

   public void enviarPorCorreo(String emailCliente) {
      System.out.println("[EMAIL] Enviando catalogo actualizado a: " + emailCliente);
      for (Producto p : productos) {
         System.out.println(" - " + p.getDescripcion());
      }
   }

   public List<Producto> getProductos() { return productos; }
}
```

EstadoOrden.java

```java
package Ejercicio_1_Televentas.com.televentas;

public enum EstadoOrden {
   INICIADA, PAGADA, ARMADA, ENVIADA, CANCELADA
}
```

IServicioInventario.java

```java
package Ejercicio_1_Televentas.com.televentas;

import java.util.List;

public interface IServicioInventario {
   Producto consultarProducto(String codigo);
   void actualizarStock(String codigo, int nuevaCantidad);
   List<Producto> obtenerCatalogo();
}
```

LogisticaService.java

```java
package Ejercicio_1_Televentas.com.televentas;

public class LogisticaService {
   public void asignarTransporte(OrdenCompra orden) {
```

```
        if (orden.getEstado() == EstadoOrden.ARMADA) {
            System.out.println("[LOGISTICA] Asignando transporte para la orden de: " +
orden.getCliente());
            orden.setEstado(EstadoOrden.ENVIADA);
        }
    }
}
```

MetodoPago.java

```
package Ejercicio_1_Televentas.com.televentas;

public interface MetodoPago {
    boolean procesar(double monto);
}
```

OrdenCompra.java

```
package Ejercicio_1_Televentas.com.televentas;

import java.util.ArrayList;
import java.util.List;

public class OrdenCompra {
    private String cliente;
    private List<Producto> items = new ArrayList<>();
    private EstadoOrden estado;
    private MetodoPago pago;

    public OrdenCompra(String cliente, MetodoPago pago) {
        this.cliente = cliente;
        this.pago = pago;
        this.estado = EstadoOrden.INICIADA;
    }

    public void agregarProducto(Producto p) {
        if (estado == EstadoOrden.INICIADA) {
            items.add(p);
        }
    }

    public double obtenerTotal() {
        return items.stream().mapToDouble(Producto::getPrecio).sum();
    }

    public void setEstado(EstadoOrden estado) {
        this.estado = estado;
    }
```

```
    public EstadoOrden getEstado() {
        return estado;
    }

    public String getCliente() {
        return cliente;
    }

    public void procesarOrden() {
        double total = obtenerTotal();
        if (pago.procesar(total)) {
            this.estado = EstadoOrden.PAGADA;
            System.out.println("[SISTEMA] Pago procesado exitosamente para " + cliente);
        }
    }

}
```

PagoTarjeta.java

```
package Ejercicio_1_Televentas.com.televentas;

public class PagoTarjeta implements MetodoPago {
    private String numero;

    public PagoTarjeta(String numero) {
        this.numero = numero;
    }

    @Override
    public boolean procesar(double monto) {
        String ultimosDigitos = numero.substring(numero.length() - 4);
        System.out.println("[BANCO] Pago de $" + monto + " aprobado con tarjeta ****" +
ultimosDigitos);
        return true;
    }
}
```

Producto.java

```
package Ejercicio_1_Televentas.com.televentas;

public class Producto {
    private String codigo;
    private String descripcion;
    private double precio;
    private int cantidadDisponible;

    public Producto(String codigo, String descripcion, double precio, int cantidadDisponible) {
```

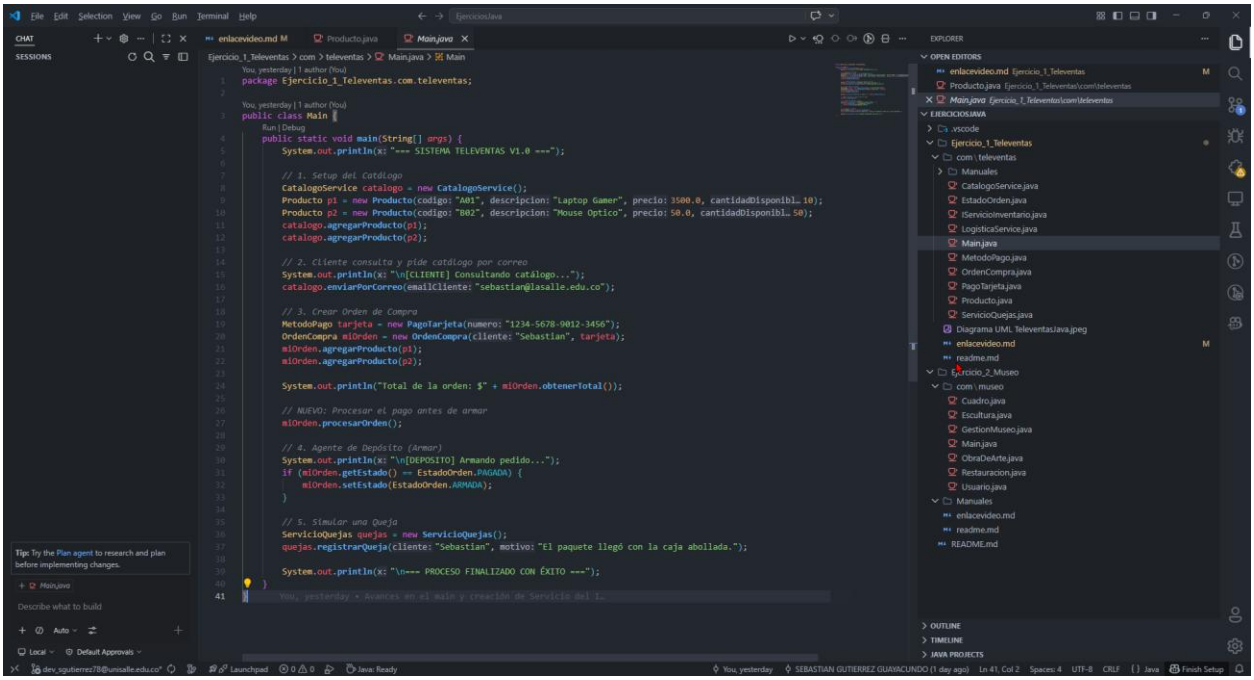
```
        this.codigo = codigo;
        this.descripcion = descripcion;
        this.precio = precio;
        this.cantidadDisponible = cantidadDisponible;
    }

    public String getCodigo() { return codigo; }
    public String getDescripcion() { return descripcion; }
    public double getPrecio() { return precio; }
    public int getCantidadDisponible() { return cantidadDisponible; }

    @Override
    public String toString() {
        return String.format("[%s] %s - $%.2f (Disponibles: %d)",
                             codigo, descripcion, precio, cantidadDisponible);
    }
}
```

ServicioQuejas.java

```
package Ejercicio_1_Televentas.com.televentas;

public class ServicioQuejas {
    public void registrarQueja(String cliente, String motivo) {
        System.out.println("[GERENCIA] Queja recibida de " + cliente + ": " + motivo);
        System.out.println("[SISTEMA] Queja remitida inmediatamente al Gerente de Relaciones.");
    }
}
```

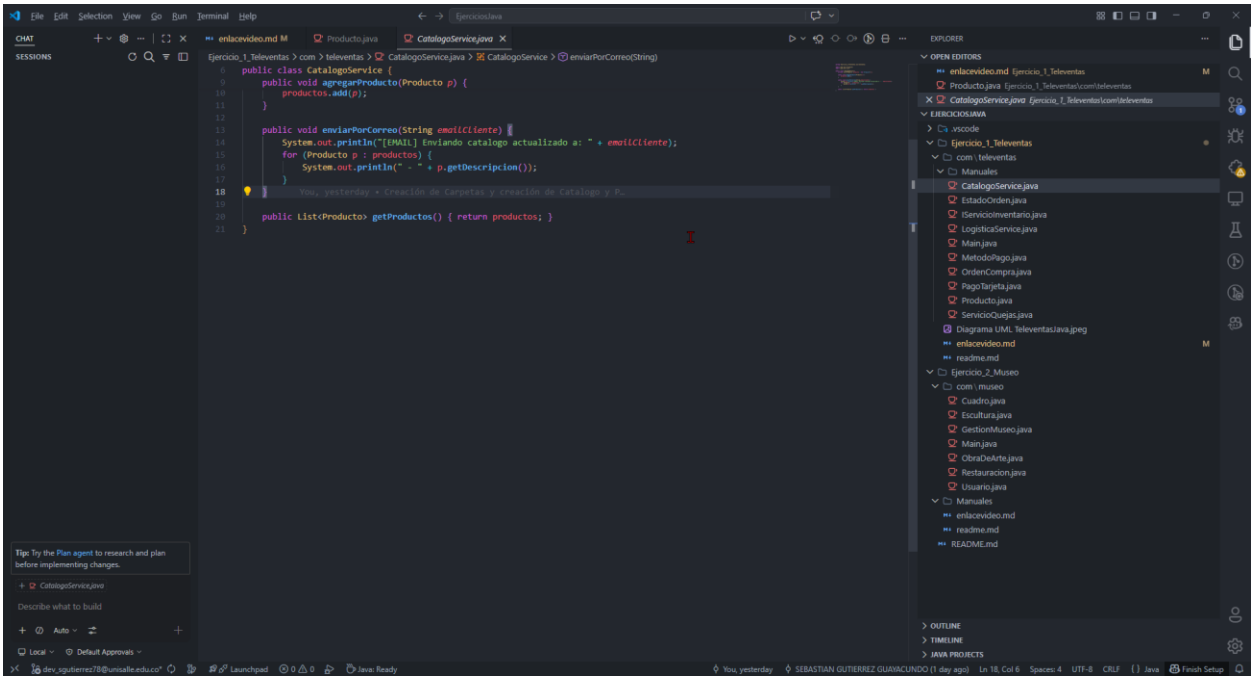
6. Código en Visual Studio Code

Main.java



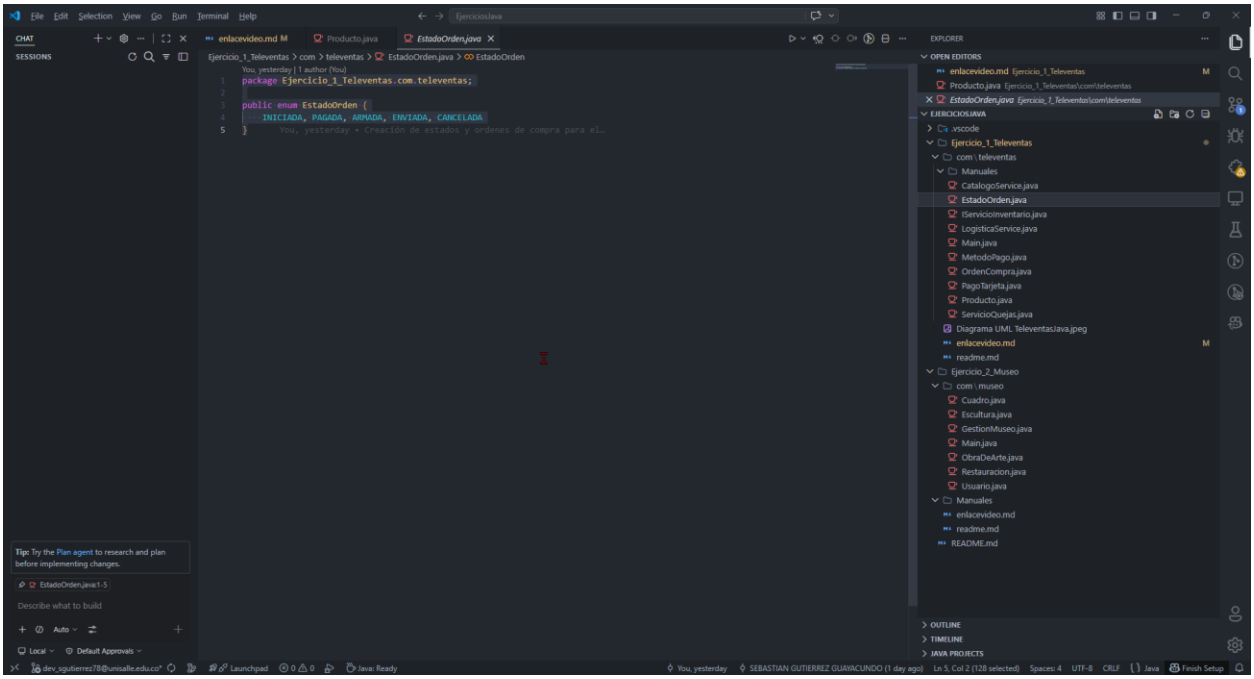
```java
package Ejercicio_1_Televentas.com.televentas;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        System.out.println(x: "=== SISTEMA TELEVENTAS V1.0 ===");

        // 1. Setup del Catálogo
        CatalogoService catalogo = new CatalogoService();
        Producto p1 = new Producto(codigo: "A01", descripcion: "Laptop Gamer", precio: 3500.0, cantidadDisponibl… 10);
        Producto p2 = new Producto(codigo: "B02", descripcion: "Mouse Optico", precio: 50.0, cantidadDisponibl… 50);
        catalogo.agregarProducto(p1);
        catalogo.agregarProducto(p2);

        // 2. Cliente consulta y pide catálogo por correo
        System.out.println(x: "\n[CLIENTE] Consultando catálogo...");
        catalogo.enviarPorCorreo(emailCliente: "sebastian@lasalle.edu.co");

        // 3. Crear Orden de Compra
        MetodoPago tarjeta = new PagoTarjeta(numero: "1234-5678-9012-3456");
        OrdenCompra miOrden = new OrdenCompra(cliente: "Sebastian", tarjeta);
        miOrden.agregarProducto(p1);
        miOrden.agregarProducto(p2);

        System.out.println("Total de la orden: $" + miOrden.obtenerTotal());

        // NUEVO: Procesar el pago antes de armar
        miOrden.procesarOrden();

        // 4. Agente de Depósito (Armar)
        System.out.println(x: "\n[DEPOSITO] Armando pedido...");
        if (miOrden.getEstado() == EstadoOrden.PAGADA) {
            miOrden.setEstado(EstadoOrden.ARMADA);
        }

        // 5. Simular una Queja
        ServicioQuejas quejas = new ServicioQuejas();
        quejas.registrarQueja(cliente: "Sebastian", motivo: "El paquete llegó con la caja abollada.");

        System.out.println(x: "\n=== PROCESO FINALIZADO CON ÉXITO ===");
    }
}
```

CatalogoService.java

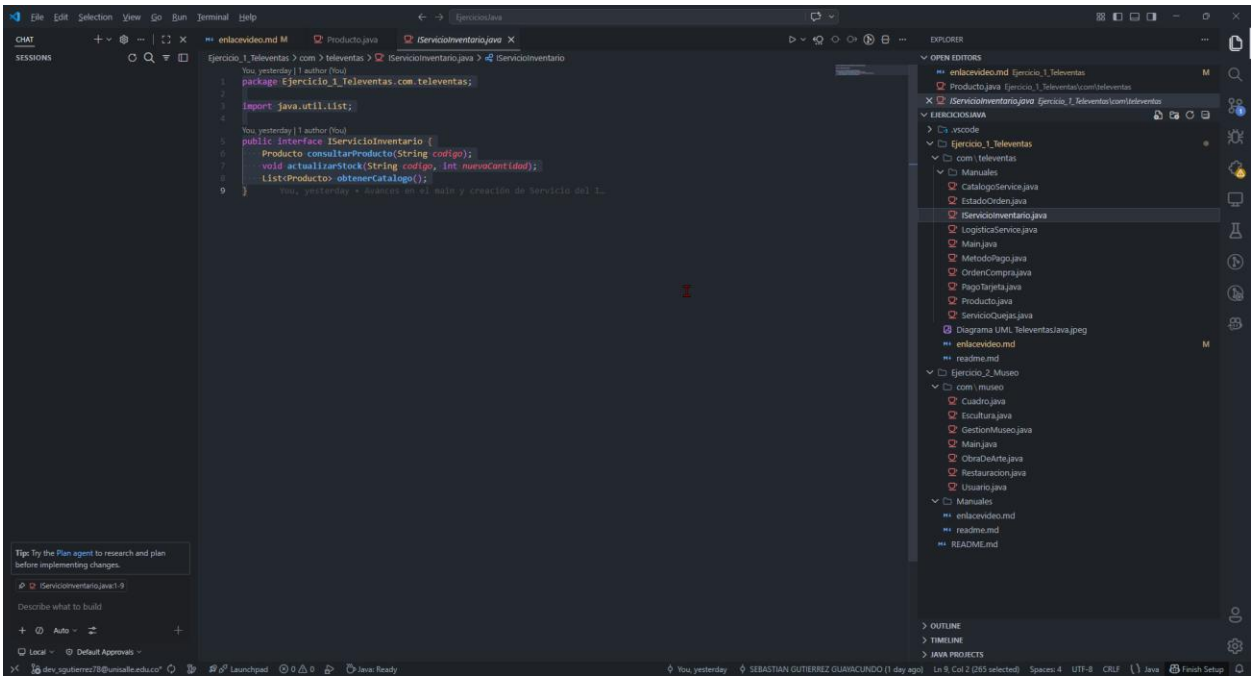


```java
public class CatalogoService {
    public void agregarProducto(Producto p) {
        productos.add(p);
    }

    public void enviarPorCorreo(String emailCliente) {
        System.out.println("[EMAIL] Enviando catalogo actualizado a: " + emailCliente);
        for (Producto p : productos) {
            System.out.println(" - " + p.getDescripcion());
        }
    }

    public List<Producto> getProductos() { return productos; }
}
```

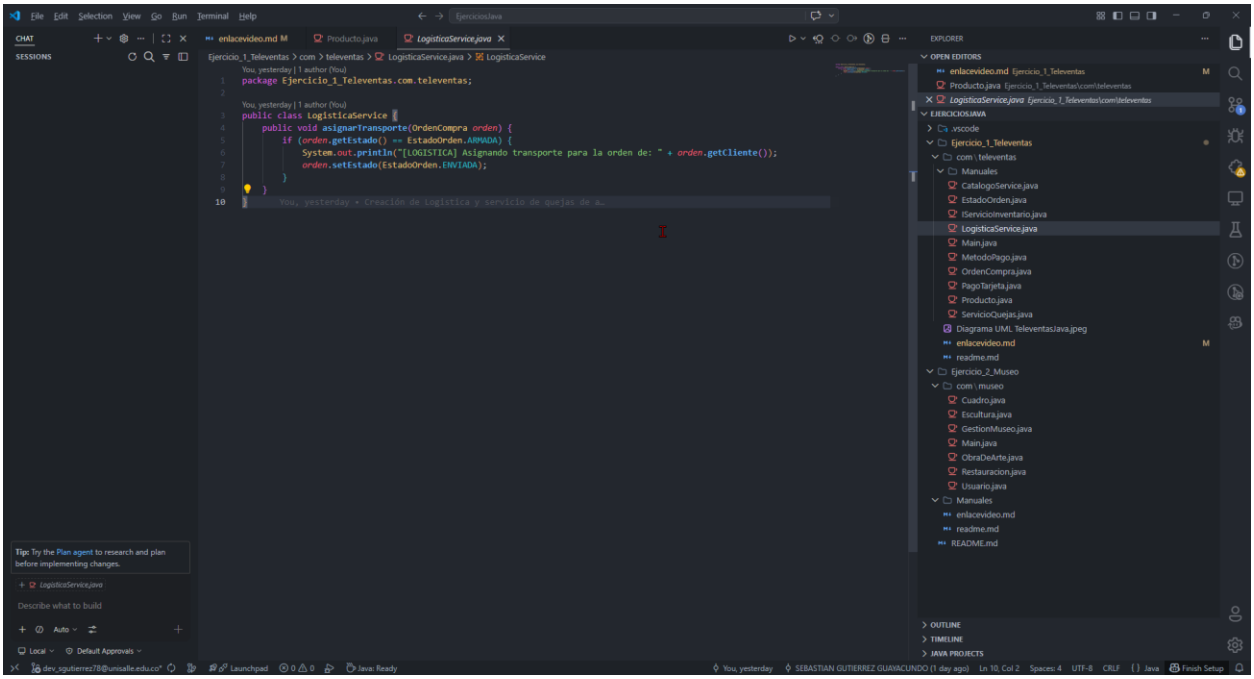
EstadoOrden.java



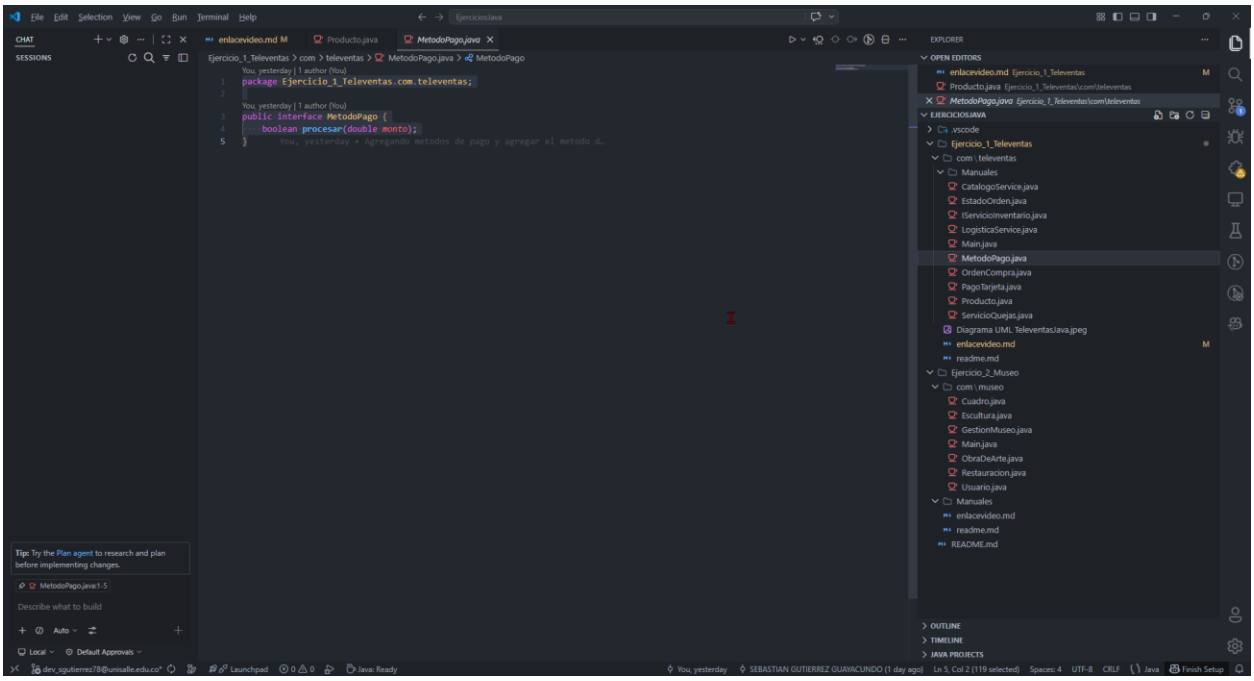
IServicioInventario.java



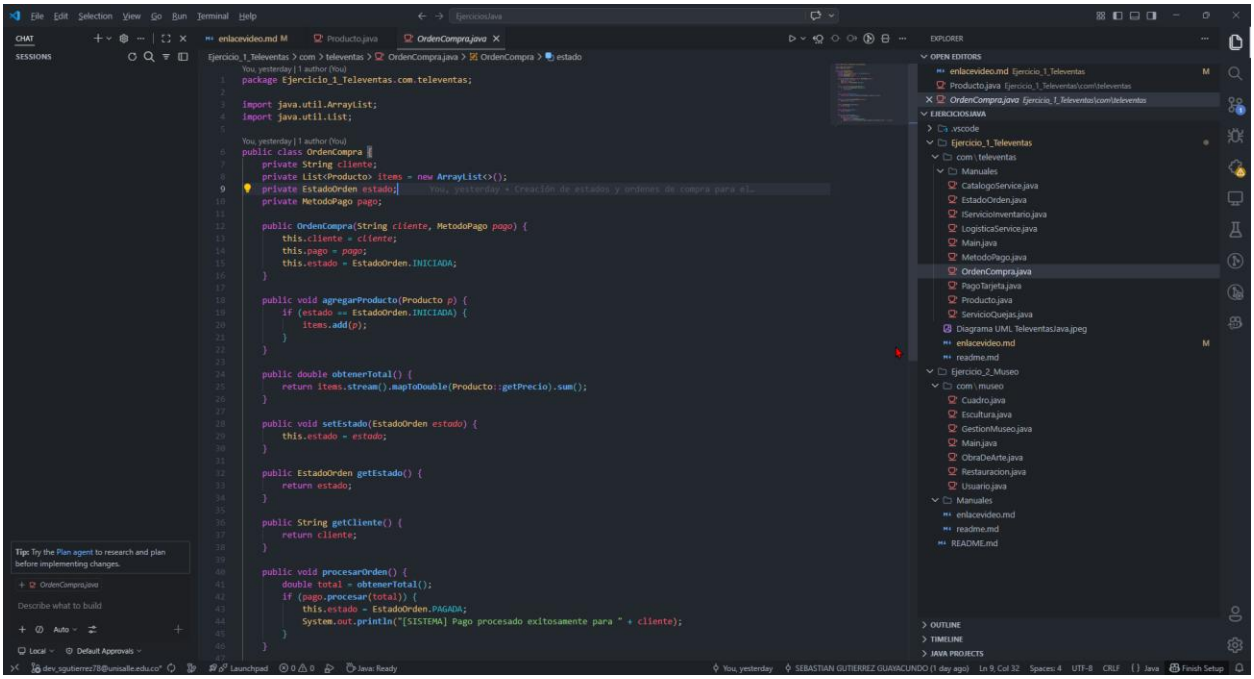
LogisticaService.java

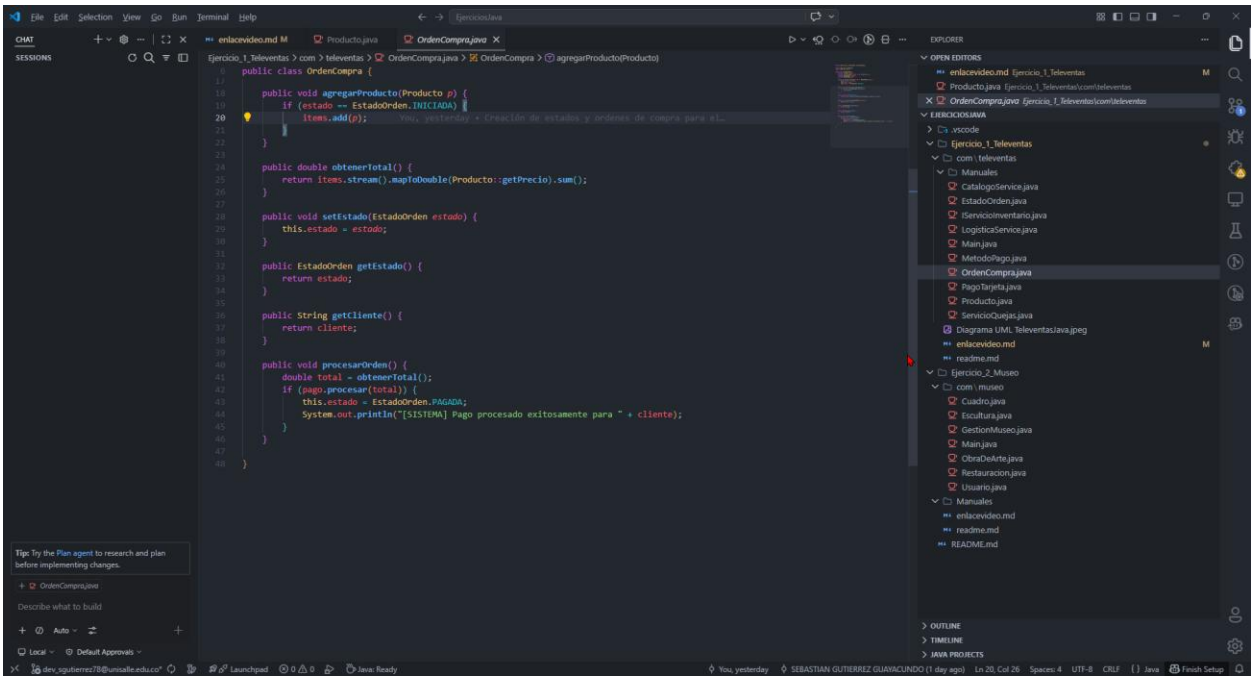


MetodoPago.java

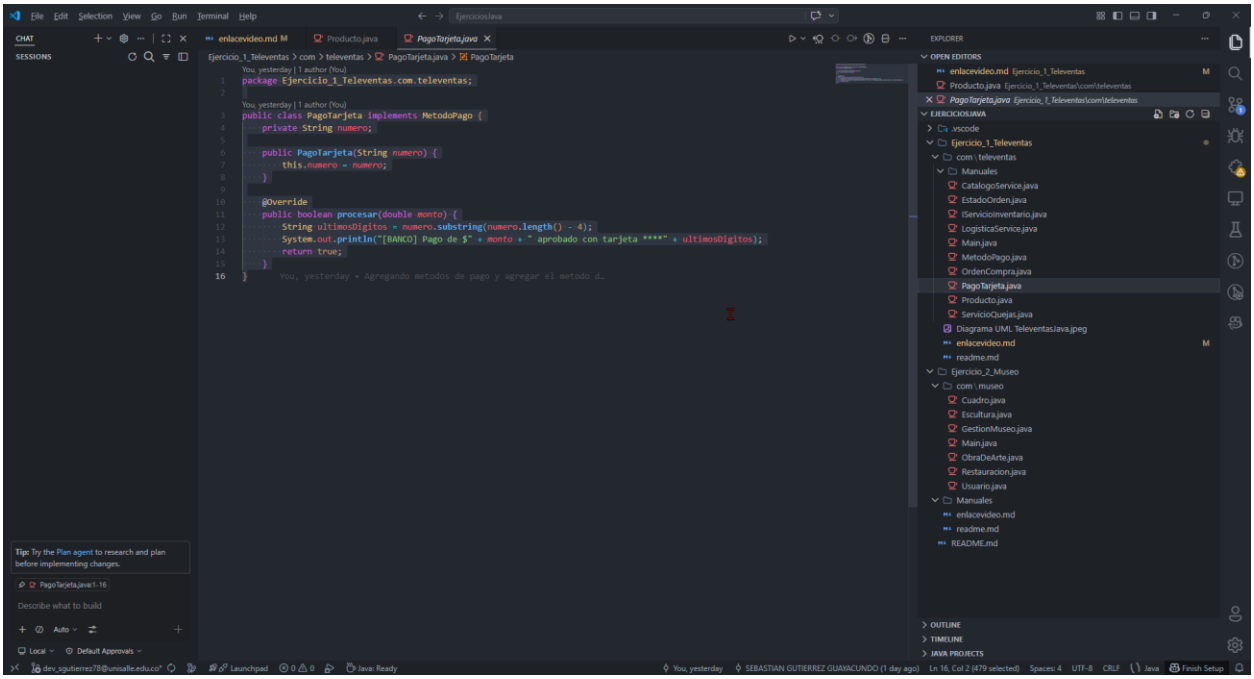


OrdenCompra.java



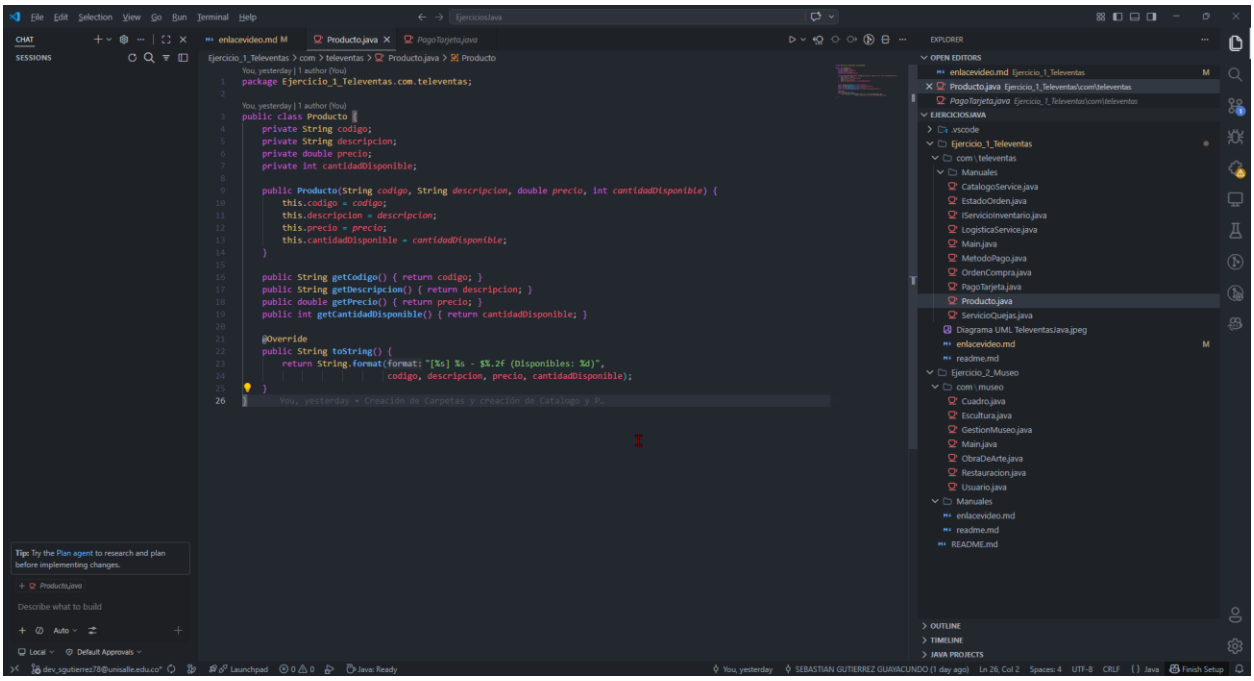


PagoTarjeta.java



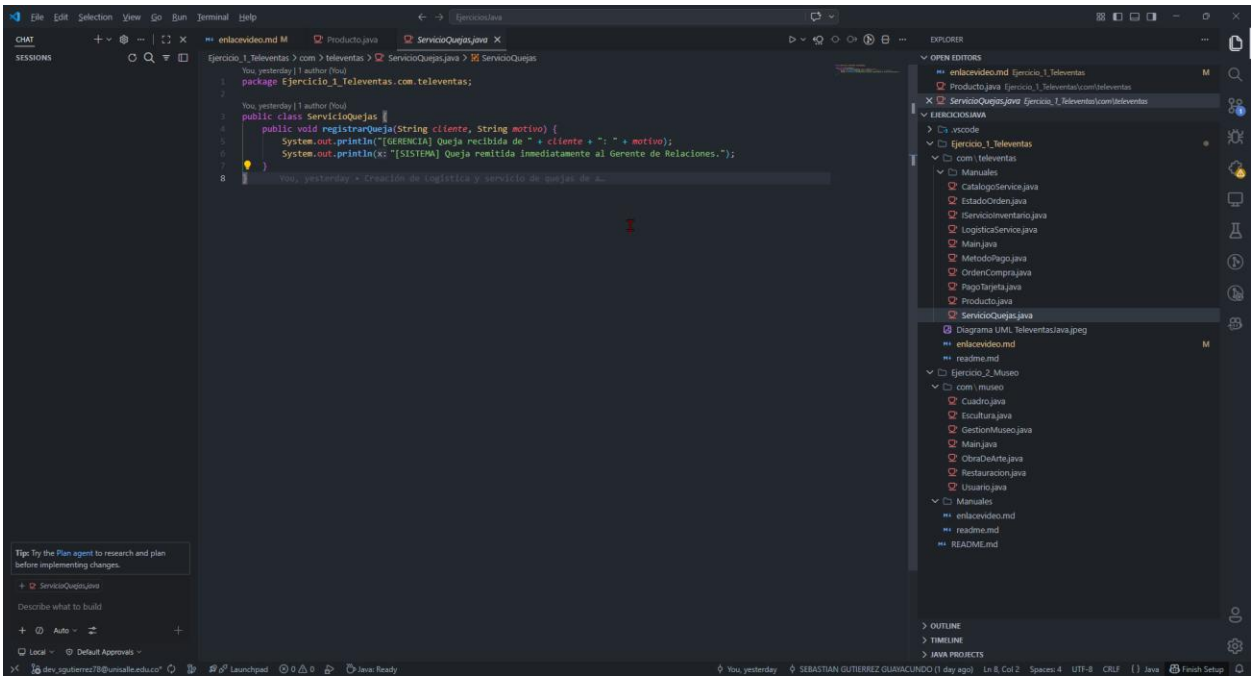
```java
package Ejercicio_1_Televentas.com.televentas;

public class PagoTarjeta implements MetodoPago {
    private String numero;

    public PagoTarjeta(String numero) {
        this.numero = numero;
    }

    @Override
    public boolean procesar(double monto) {
        String ultimosDigitos = numero.substring(numero.length() - 4);
        System.out.println("[BANCO] Pago de $" + monto + " aprobado con tarjeta ****" + ultimosDigitos);
        return true;
    }
}
```

Producto.java



```java
package Ejercicio_1_Televentas.com.televentas;

public class Producto {
    private String codigo;
    private String descripcion;
    private double precio;
    private int cantidadDisponible;

    public Producto(String codigo, String descripcion, double precio, int cantidadDisponible) {
        this.codigo = codigo;
        this.descripcion = descripcion;
        this.precio = precio;
        this.cantidadDisponible = cantidadDisponible;
    }

    public String getCodigo() { return codigo; }
    public String getDescripcion() { return descripcion; }
    public double getPrecio() { return precio; }
    public int getCantidadDisponible() { return cantidadDisponible; }

    @Override
    public String toString() {
        return String.format("[%s] %s - $%.2f (Disponibles: %d)",
                             codigo, descripcion, precio, cantidadDisponible);
    }
}
```

ServicioQuejas.java



7. Prueba de Escritorio

Resultado:

```
sebas@SoyySebas MINGW64 ~/EjerciciosJava (dev_sgutierrez78@unisalle.edu.co)
$  cd c:\\Users\\sebas\\EjerciciosJava ; /usr/bin/env C:\\Program\ Files\\Java\\jdk-24\\bin\\java.exe -XX:+ShowCodeDetailsInExceptionMess
ages -cp C:\\Users\\sebas\\AppData\\Roaming\\Code\\User\\workspaceStorage\\e690b0e808913fa901f38a0132a8d4d9\\redhat.java\\jdt_ws\\Ejercic
iosJava_d3a065d0\\bin Ejercicio_1_Televentas.com.televentas.Main
=== SISTEMA TELEVENTAS V1.0 ===

[CLIENTE] Consultando catálogo...
[EMAIL] Enviando catalogo actualizado a: sebastian@lasalle.edu.co
 - Laptop Gamer
 - Mouse Optico
Total de la orden: $3550.0
[BANCO] Pago de $3550.0 aprobado con tarjeta ****3456
[SISTEMA] Pago procesado exitosamente para Sebastian

[DEPOSITO] Armando pedido...
[GERENCIA] Queja recibida de Sebastian: El paquete llegó con la caja abollada.
[SISTEMA] Queja remitida inmediatamente al Gerente de Relaciones.

=== PROCESO FINALIZADO CON ÉXITO ===

sebas@SoyySebas MINGW64 ~/EjerciciosJava (dev_sgutierrez78@unisalle.edu.co)
$
```

Tras la ejecución de la simulación y el análisis de la trazabilidad en consola, se pueden extraer las siguientes conclusiones:

- **Integridad del Proceso de Negocio:** El sistema garantiza un flujo secuencial lógico. Como se observa en el log, la Logística solo se activa después de que el Depósito confirma el

empaque, y este a su vez solo actúa tras una Transacción exitosa. Esto demuestra un control efectivo de los estados del objeto OrdenCompra.

- **Gestión Eficiente de Inventario:** El mensaje [Orden] Agregado confirma que la comunicación entre la clase OrdenCompra y Producto es bidireccional, permitiendo la reserva inmediata del stock para evitar ventas en exceso (overselling).

- **Desacoplamiento de Soporte (Escalabilidad):** El sistema de Quejas funciona de manera independiente al flujo de despacho. El resultado muestra que, aunque el pedido ya fue entregado a la transportadora, el módulo de soporte puede capturar incidencias y remitirlas a la Gerencia sin afectar la base de datos de ventas o logística.

- **Eficacia de la Abstracción:** El uso de una clase abstracta para el pago permitió que el sistema procesara la Tarjeta de Crédito de forma transparente, validando que la arquitectura es flexible para integrar nuevos métodos de pago en futuras fases del proyecto.